

Jutsu Master
Computer Vision and iOS Game Documentation

Farrell Habibie Putra Haris

April 16, 2026

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Problem Statement	1
1.3	Technical Objectives	1
1.4	Unified Feature Design	2
1.5	End-to-End Runtime Flow	2
1.6	Document Scope	2
2	Dataset	3
2.1	Source and Annotation Format	3
2.2	Data Splits	3
2.3	Class Taxonomy	3
2.4	Class Distribution (Exported Unified Dataset)	4
2.5	Label Alias Note	4
2.6	Attached Asset Placeholders	4
3	Model Training	7
3.1	Notebook Scope	7
3.2	Preprocessing and Feature Extraction	7
3.3	Training Configuration	8
3.4	Exported Artifacts	8
3.5	Evaluation Metrics	8
3.6	Confusion Matrix Notes	9
4	iOS Architecture	10
4.1	Application Structure	10
4.2	Core Runtime Managers	10
4.3	Model and Backend Fallback Strategy	10
4.4	Frame-to-Action Pipeline	11
4.5	Rendering Stack	11
5	Gameplay	13
5.1	Game Modes	13
5.2	Sign Sequence Logic	13
5.3	Battle Mechanics	13
5.4	Visual and Audio Feedback	14
5.5	Attached Asset Placeholders	14
5.6	App Screenshot Placeholders	16

6	Results	19
6.1	Quantitative Summary	19
6.2	Model Behavior Observations	19
6.3	iOS Runtime Observations	19
6.4	Runtime Screenshot Placeholders	20
7	Limitations and Future Work	22
7.1	Current Limitations	22
7.1.1	Dataset and Labeling	22
7.1.2	Model and Inference	22
7.1.3	iOS Runtime and UX	22
7.2	Recommended Improvements	22
7.3	Documentation Maintenance Plan	23
A	Expected Screenshot Files	24

Chapter 1

Introduction

1.1 Project Overview

Jutsu Master is a real-time computer vision application that detects Naruto hand signs from live camera input, classifies gestures with a CoreML model, and maps valid sign sequences into interactive jutsu effects and gameplay states on iOS.

The full stack combines:

- dataset preparation and training in Python,
- feature extraction with MediaPipe hand landmarks,
- RandomForest classification exported to CoreML,
- iOS runtime inference, sequence logic, visual effects, and audio.

1.2 Problem Statement

Hand-sign recognition in real-world camera conditions is difficult because of partial visibility, one-hand versus two-hand transitions, and pose variation between users. The project targets a robust unified model that can operate even when only one hand is visible.

1.3 Technical Objectives

- Build a single feature representation for one-hand and two-hand gestures.
- Train a deployable classifier for 12 hand-sign classes.
- Export and integrate the model into an iOS app pipeline.
- Implement sequence-based jutsu detection and game logic.
- Deliver a playable experience across free, tutorial, speed, and battle modes.

1.4 Unified Feature Design

Each hand contributes 21 landmarks with x, y, z coordinates.

$$d = 2 \times 21 \times 3 = 126$$

The model input vector is:

$$\mathbf{x} = [\mathbf{h}_{\text{left}}, \mathbf{h}_{\text{right}}] \in \mathbb{R}^{126}$$

If only one hand is detected, the missing side is filled with 63 zeros. This preserves a fixed input dimension while retaining one-hand usability.

1.5 End-to-End Runtime Flow

1. Capture a camera frame.
2. Detect hand landmarks (MediaPipe or Vision fallback).
3. Normalize and merge features into a 126-dimensional vector.
4. Run CoreML gesture inference.
5. Apply hold logic and sequence tracking.
6. Trigger jutsu effect, audio, and mode-specific game state updates.

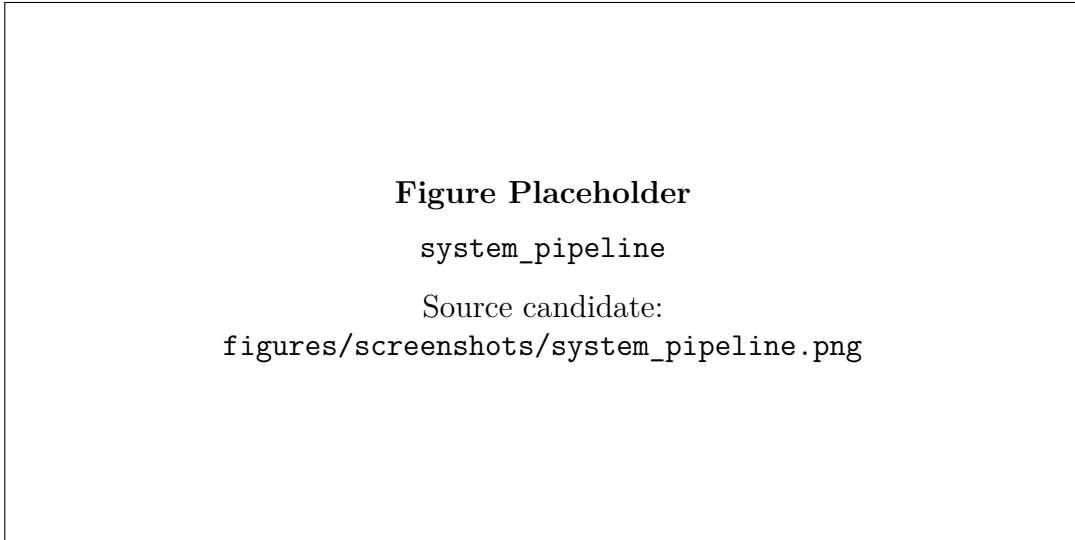


Figure 1.1: System pipeline placeholder. Replace with a final architecture diagram or app pipeline screenshot.

1.6 Document Scope

This report documents dataset preparation, model training, iOS architecture, gameplay logic, measured results, and current limitations, with figure placeholders for assets and screenshots.

Chapter 2

Dataset

2.1 Source and Annotation Format

The training dataset is based on YOLO-style labels where each annotation stores class index and normalized bounding box coordinates.

Primary dataset configuration:

- Location: `hand_gesture_model/naruto-hand-sign-4/`
- Config file: `hand_gesture_model/naruto-hand-sign-4/data.yaml`
- Number of classes: 12
- Class names: bird, boar, dog, dragon, hare, horse, monkey, ox, ram, rat, snake, tiger

2.2 Data Splits

The dataset uses train, validation, and test folder splits:

- `train/images` and `train/labels`
- `valid/images` and `valid/labels`
- `test/images` and `test/labels`

2.3 Class Taxonomy

Table 2.1: Hand-sign classes from dataset metadata

Class count	12
Sign labels	bird, boar, dog, dragon, hare, horse, monkey, ox, ram, rat, snake, tiger

2.4 Class Distribution (Exported Unified Dataset)

Table 2.2: Class counts from `class_distribution_unified.csv`

Class	Count	Class	Count
bird	124	monkey	562
boar	105	ox	302
dog	368	ram	370
dragon	367	rat	259
hare	118	snake	280
horse	455	tiger	651

2.5 Label Alias Note

In iOS gameplay logic, *hare* is normalized to *rabbit* for asset naming and sequence consistency. This alias handling should be kept explicit in deployment documentation.

2.6 Attached Asset Placeholders

The following placeholders correspond to attached sign and reference assets.

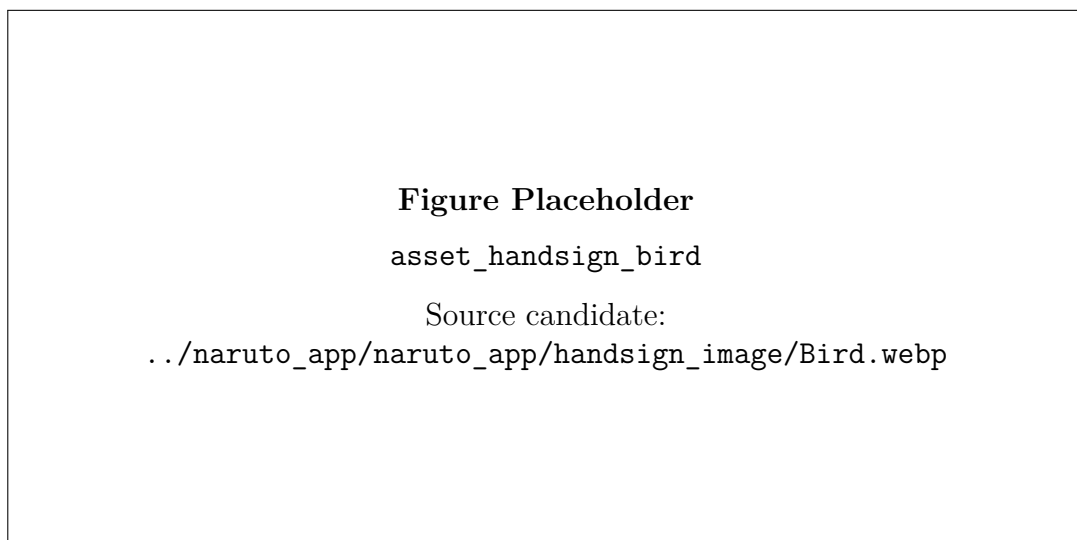


Figure 2.1: Bird hand-sign reference asset placeholder.

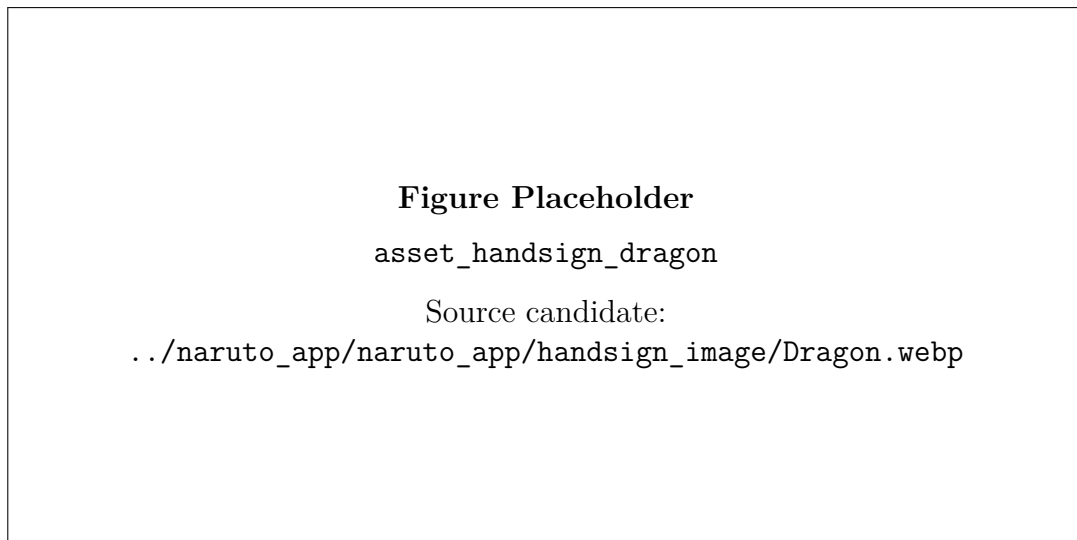


Figure 2.2: Dragon hand-sign reference asset placeholder.

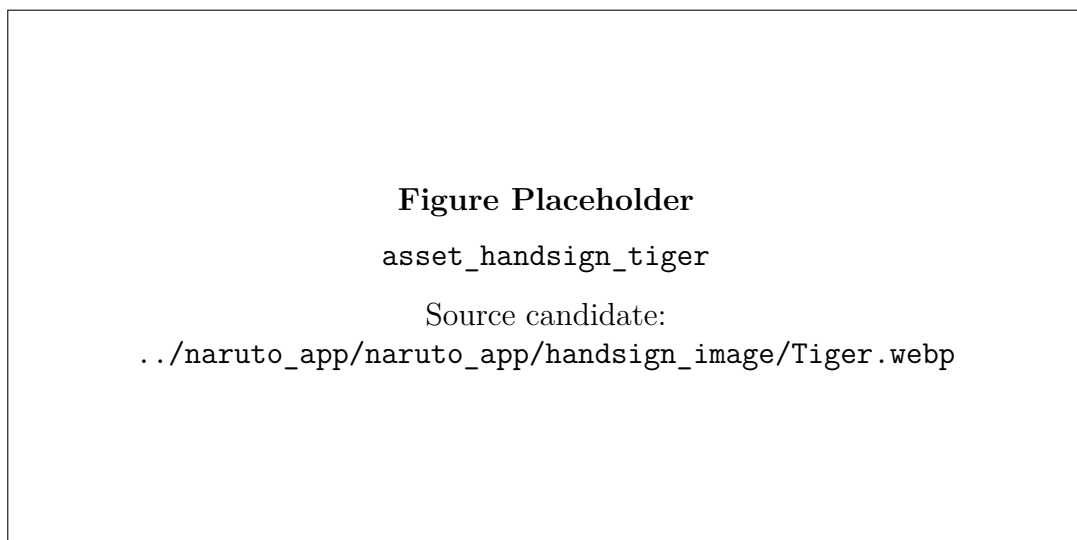


Figure 2.3: Tiger hand-sign reference asset placeholder.

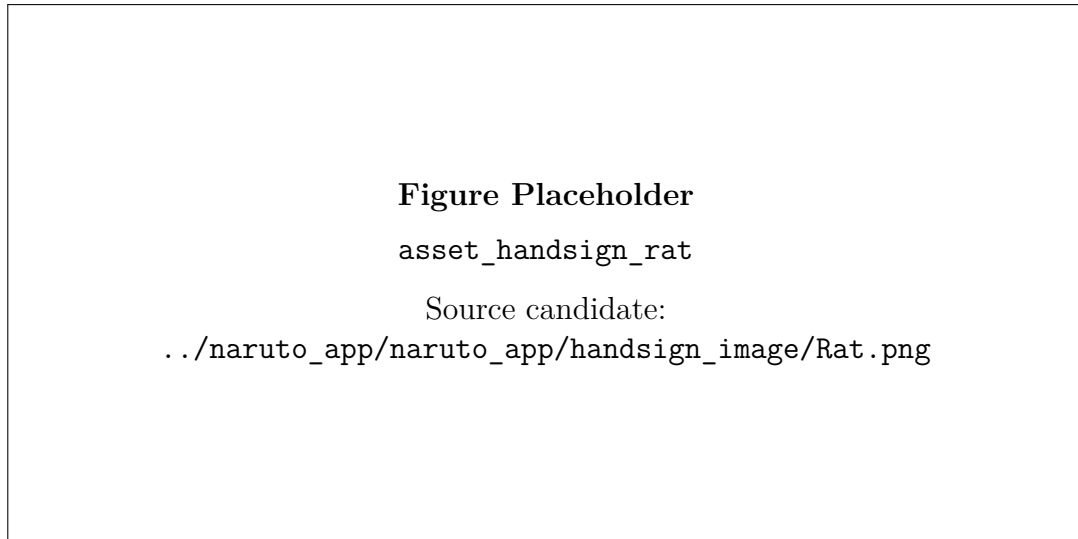


Figure 2.4: Rat hand-sign reference asset placeholder.

Chapter 3

Model Training

3.1 Notebook Scope

Training is implemented in:

- `hand_gesture_model/train2.ipynb`

This notebook builds a unified one-hand and two-hand classifier with fixed 126-feature input vectors.

3.2 Preprocessing and Feature Extraction

For each labeled bounding box:

1. Read YOLO box and crop hand region.
2. Detect up to two hands with MediaPipe.
3. Normalize each hand relative to wrist landmark.
4. Scale landmarks by the maximum 2D norm.
5. Assign features to left and right slots.
6. Zero-pad missing hand slot when only one hand is present.

Normalization equation per hand:

$$\hat{\mathbf{p}}_i = \mathbf{p}_i - \mathbf{p}_0, \quad s = \max_j \|\hat{\mathbf{p}}_{j,xy}\|_2, \quad \tilde{\mathbf{p}}_i = \frac{\hat{\mathbf{p}}_i}{\max(s, 10^{-6})}$$

Final feature vector:

$$\mathbf{x} = [\tilde{\mathbf{p}}_{L,1..21}, \tilde{\mathbf{p}}_{R,1..21}] \in \mathbb{R}^{126}$$

3.3 Training Configuration

- Model: RandomForestClassifier
- Trees: 400
- Class balancing: `balanced_subsample`
- Split: 80:20 stratified train-test
- Random seed: 42

3.4 Exported Artifacts

Unified pipeline outputs are written to:

- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/hand_landmarks_data.csv`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/X_landmarks_unified.npy`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/y_labels_unified.npy`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/confusion_matrix_unified.csv`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/hand_gesture_rf_unified.pkl`
- `hand_gesture_model/naruto-hand-sign-4/phase1_outputs_unified/training_summary_unified.csv`

3.5 Evaluation Metrics

Table 3.1: Unified model extraction and evaluation metrics

Metric	Value
Usable samples	3961
One-hand samples	3289 (83.03%)
Two-hand samples	672 (16.97%)
Skipped images	2181
Skipped boxes	0
Skipped no hands	2177
Feature dimension	126
Train-test split	80:20 (stratified)
Estimated test samples	793
Overall test accuracy	96.85%
CoreML export	Success (<code>hand_gesture_rf_unified.mlmodel</code>)

3.6 Confusion Matrix Notes

The exported confusion matrix shows strong diagonal dominance, with minor confusion among visually similar signs (for example, dragon vs snake, and ram vs tiger under some samples).

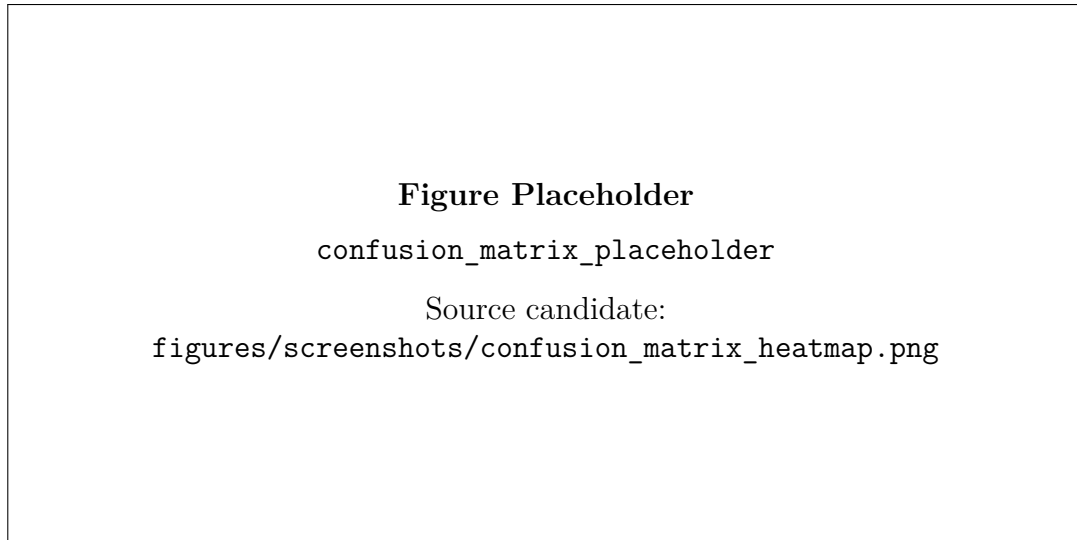


Figure 3.1: Confusion matrix heatmap placeholder generated from CSV metrics.

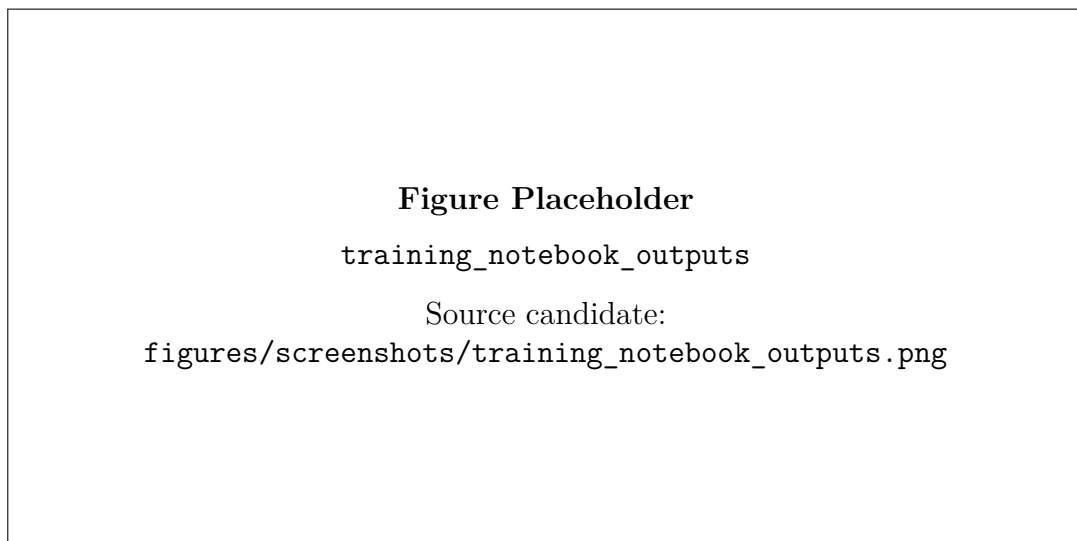


Figure 3.2: Notebook output screenshot placeholder for extraction summary and training logs.

Chapter 4

iOS Architecture

4.1 Application Structure

The iOS app is implemented in:

- `naruto_app/naruto_app/`

Top-level flow:

- `naruto_appApp.swift` launches `ContentView.swift`.
- Navigation routes users from Home to Mode Setup and then into game views.
- `CameraGameView.swift` handles free, speed, and tutorial modes.
- `BattleGameView.swift` combines camera inference with battle simulation.

4.2 Core Runtime Managers

Table 4.1: Manager responsibilities in the runtime pipeline

Component	Responsibility
CameraManager	Camera permission, session setup, frame streaming, orientation and mirroring management.
HandLandmarkDetector	Hand landmark extraction with MediaPipe Tasks and Vision fallback.
FaceDirectionEstimator	Mouth and face direction estimation for fireball and burning ash effects.
GestureRecognizer	Feature preprocessing, CoreML model inference, score formatting, top-k output.
JutsuManager	Hold logic, sequence tracking, mode-specific trigger decisions, effect state memory.

4.3 Model and Backend Fallback Strategy

- Gesture model loading attempts unified model first, then two-hand, then single-hand model.

- Landmark detection attempts MediaPipe runtime task models and falls back to Vision requests if unavailable.
- Face direction estimation similarly uses MediaPipe face mesh first with Vision fallback.

4.4 Frame-to-Action Pipeline

1. Camera delivers a sample buffer.
2. Hand and face landmarks are extracted.
3. Landmark vectors are canonicalized for mirror and orientation consistency.
4. Classifier predicts sign label and confidence.
5. JutsuManager processes hold and sequence state.
6. Active jutsu state updates overlays, particles, sounds, and game counters.

4.5 Rendering Stack

- `CameraView.swift` overlays skeletons and effects above AVCapture preview.
- `CAEmitterLayer` handles fire, lightning, and ash particle systems.
- `SpriteKit` scenes handle water dragon and kuchiyose summon animations.
- Overlay UI components display target sign guidance and progress strips.

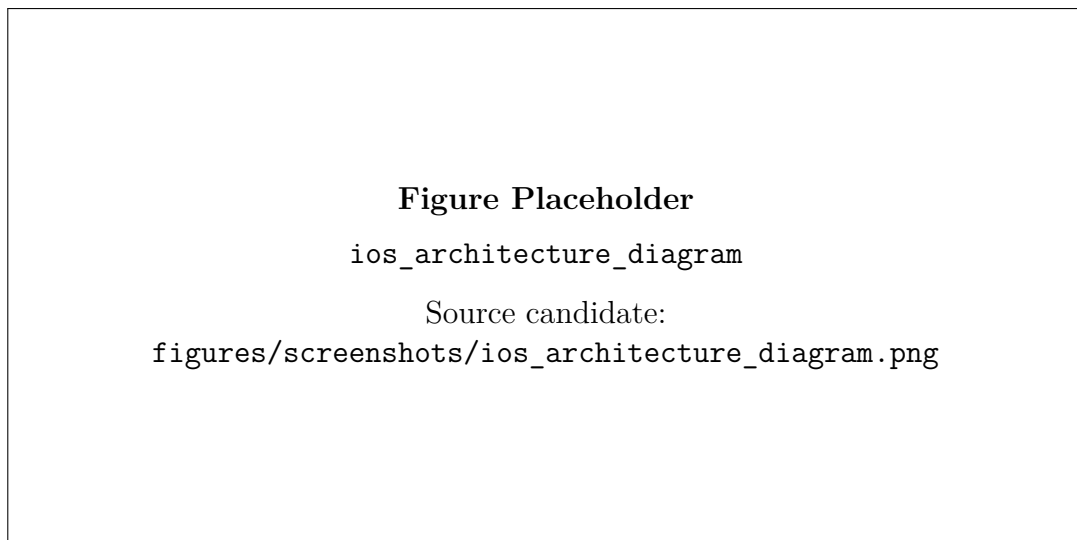


Figure 4.1: iOS architecture diagram placeholder.

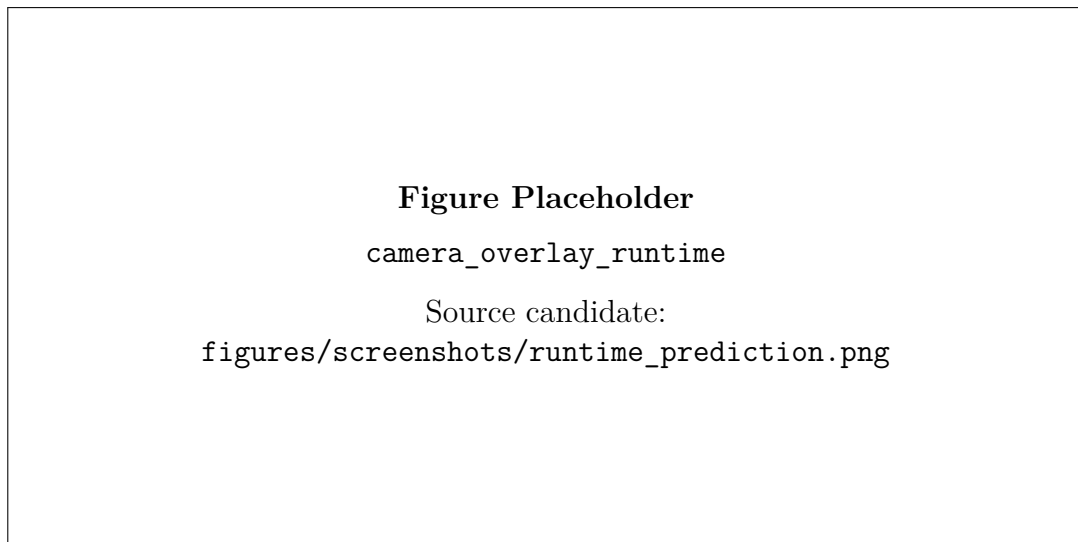


Figure 4.2: Runtime camera overlay screenshot placeholder.

Chapter 5

Gameplay

5.1 Game Modes

The app supports four user modes:

- Battle: defend and counterattack against Sasuke with HP and chakra systems.
- Free: trigger any supported jutsu without fixed target constraints.
- Speed: complete a selected jutsu sequence as fast as possible.
- Tutorial: follow guided signs for a selected target jutsu.

5.2 Sign Sequence Logic

Each recognized sign must be held before commitment, then matched against jutsu sequences. The same recognized signs can produce different outcomes depending on mode and context.

Table 5.1: Jutsu-to-sign mapping used by game logic

Jutsu	Sign sequence	Count
Fire Style: Burning Ash	dragon → ox → dog → horse	4
Fire Style: Fireball	horse → snake → monkey → boar → horse	5
Jutsu		
Fire	bird → snake	2
Kuchiyose no Jutsu	boar → horse → monkey → bird	4
Chidori	ox → monkey	2
Rasengan	monkey → bird	2
Water Style: Water	monkey → boar → bird → ox → horse → bird →	8
Dragon Bullet	dog → snake	
Wind Style: Rasengan	horse → monkey	2

5.3 Battle Mechanics

Battle mode adds a state machine around CV inference:

1. Enemy attack phase selects an incoming jutsu.
2. Player defend phase expects a specific counter jutsu sequence.
3. Player attack phase allows offensive jutsu launches.
4. Round end restores partial chakra and advances battle pacing.

Counter mapping examples:

- Fireball or Burning Ash is countered by Water Dragon.
- Lightning is countered by Rasengan.

5.4 Visual and Audio Feedback

- Overlay guidance shows current target sign and sequence progression.
- Particle and scene effects represent each jutsu style.
- Audio loops adapt volume and rate for active effects such as Rasengan and Wind.

5.5 Attached Asset Placeholders

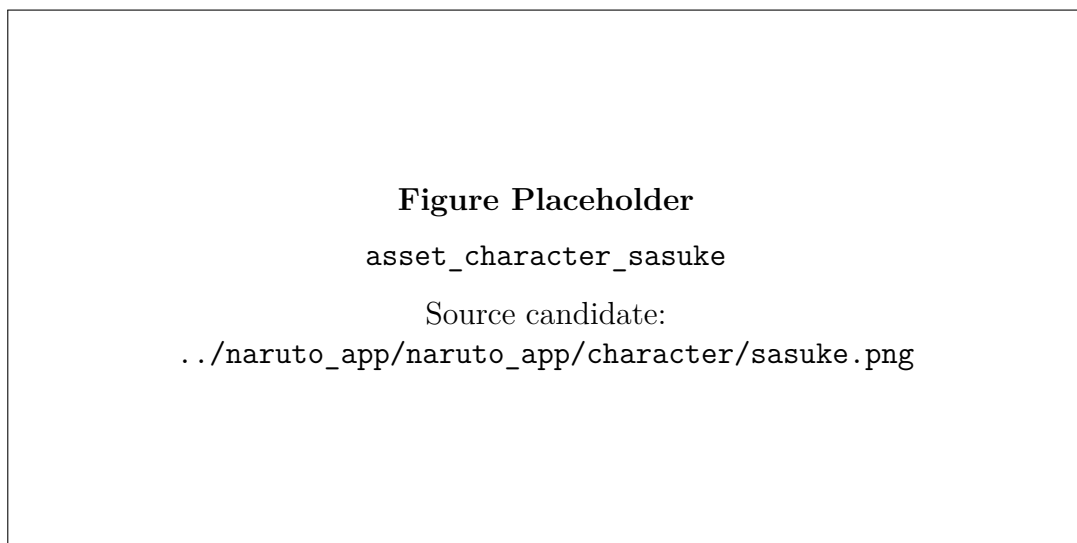


Figure 5.1: Sasuke character asset placeholder used in battle mode.

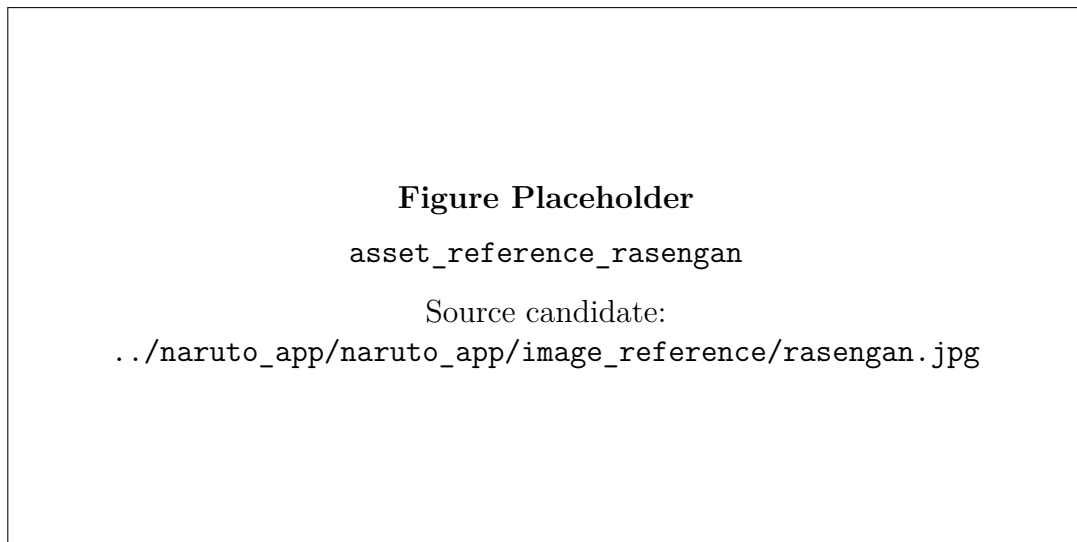


Figure 5.2: Rasengan visual reference asset placeholder.

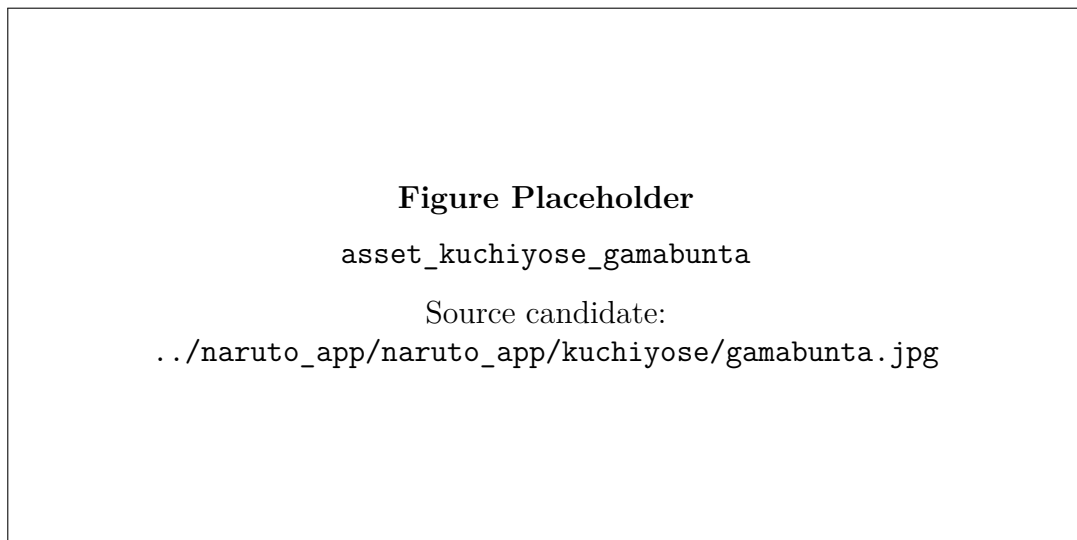


Figure 5.3: Gamabunta summon asset placeholder.

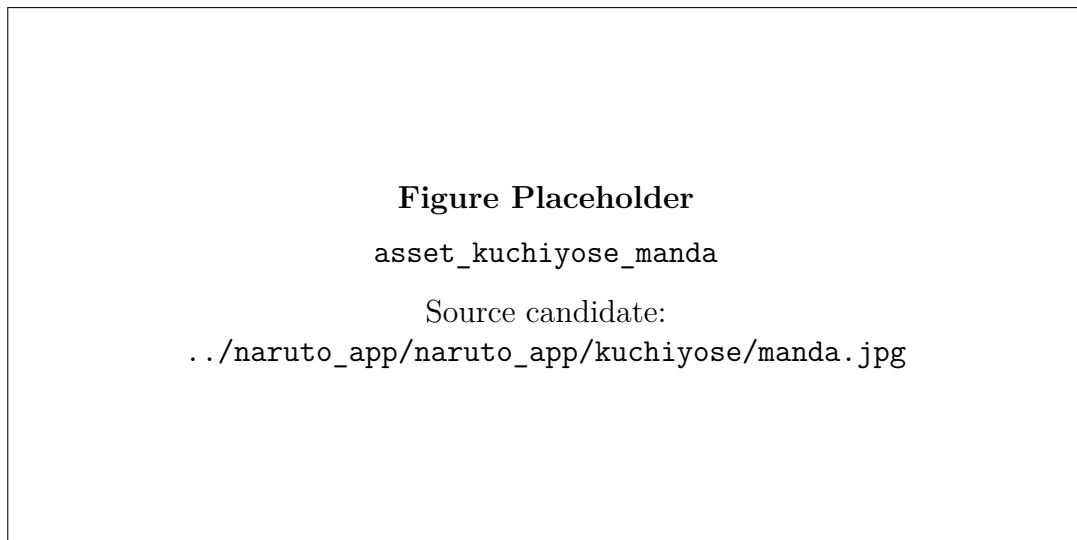


Figure 5.4: Manda summon asset placeholder.

5.6 App Screenshot Placeholders

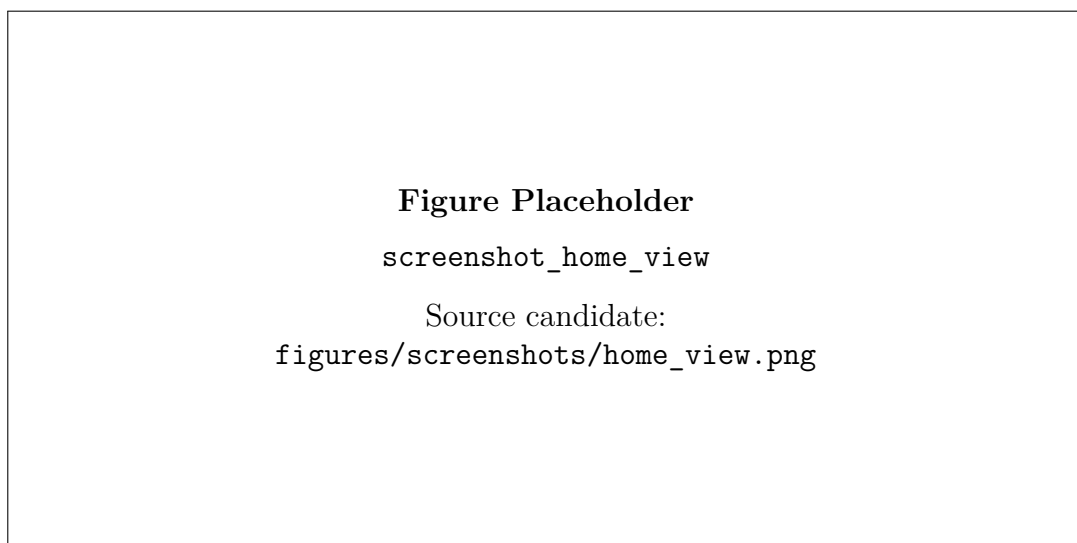


Figure 5.5: Home view screenshot placeholder.

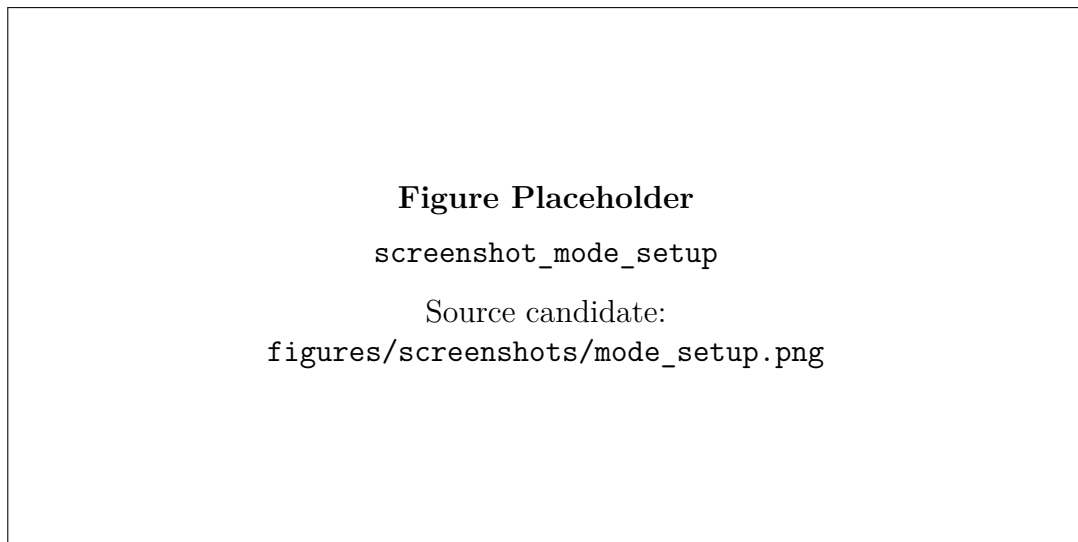


Figure 5.6: Mode setup screenshot placeholder.

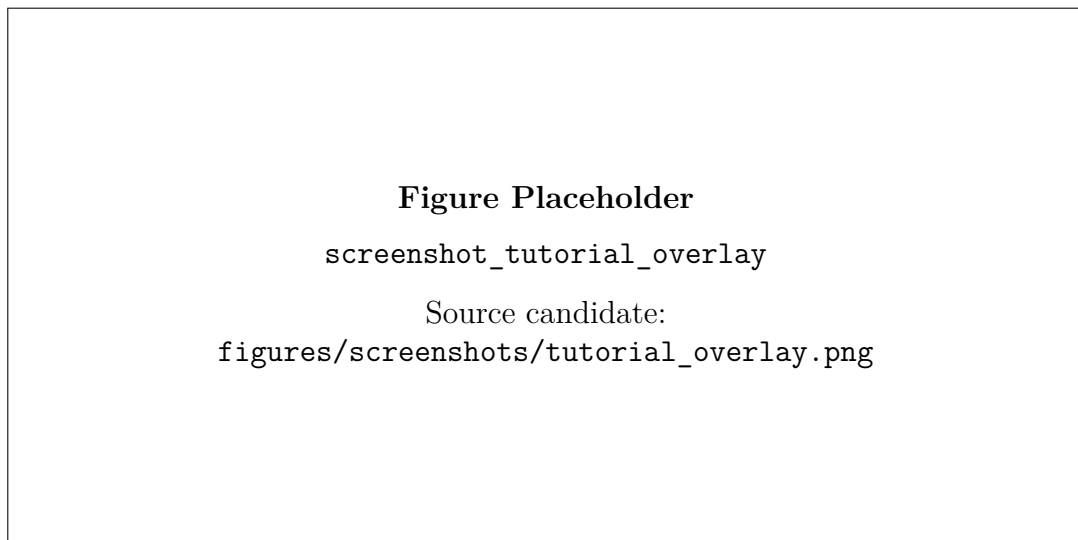


Figure 5.7: Tutorial overlay screenshot placeholder.

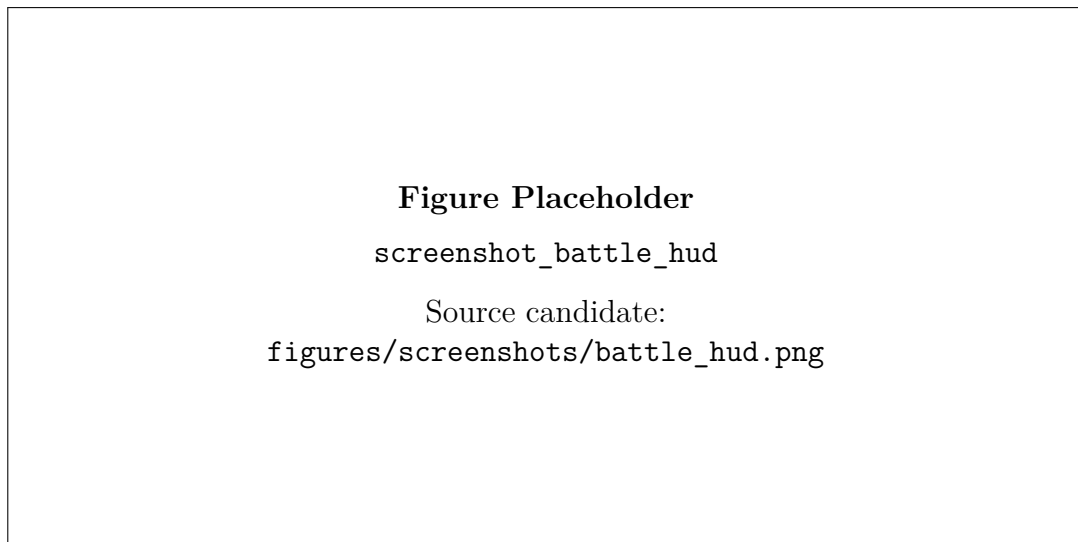


Figure 5.8: Battle HUD screenshot placeholder.

Chapter 6

Results

6.1 Quantitative Summary

The unified model achieved strong test accuracy and a successful CoreML export for on-device use.

- Test accuracy: 96.85%
- Feature dimension: 126
- Usable samples: 3961
- One-hand coverage: 83.03%
- Two-hand coverage: 16.97%

6.2 Model Behavior Observations

- Most classes are separated reliably under clean lighting and stable framing.
- Minor confusions occur among similar finger formations.
- One-hand fallback through zero padding improves practical robustness.

6.3 iOS Runtime Observations

- Real-time inference supports responsive sequence gameplay.
- Combined hand and face cues improve fire-style effect direction logic.
- Battle mode adds meaningful interaction through defend and attack windows.

6.4 Runtime Screenshot Placeholders

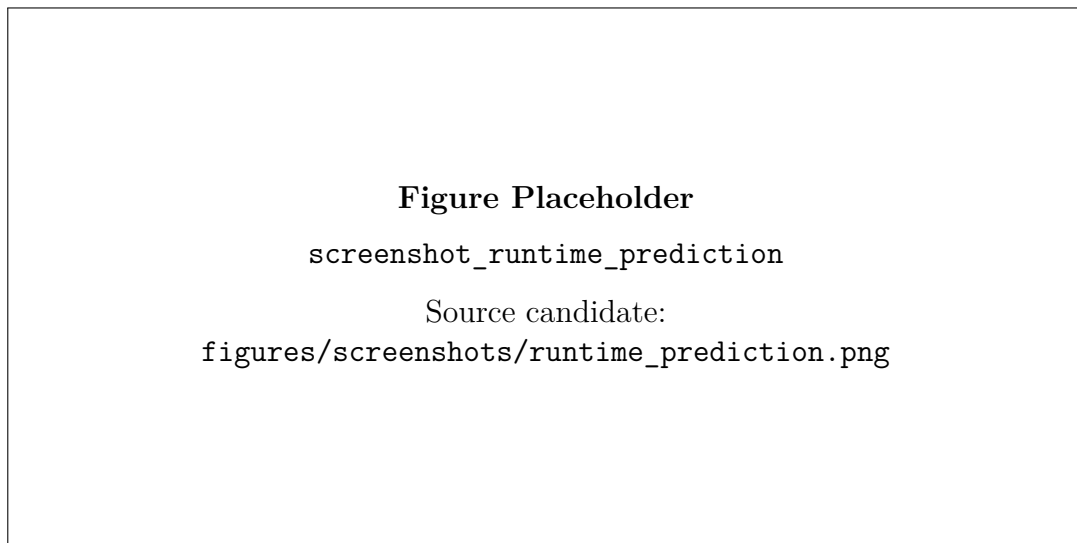


Figure 6.1: Runtime sign prediction screenshot placeholder.

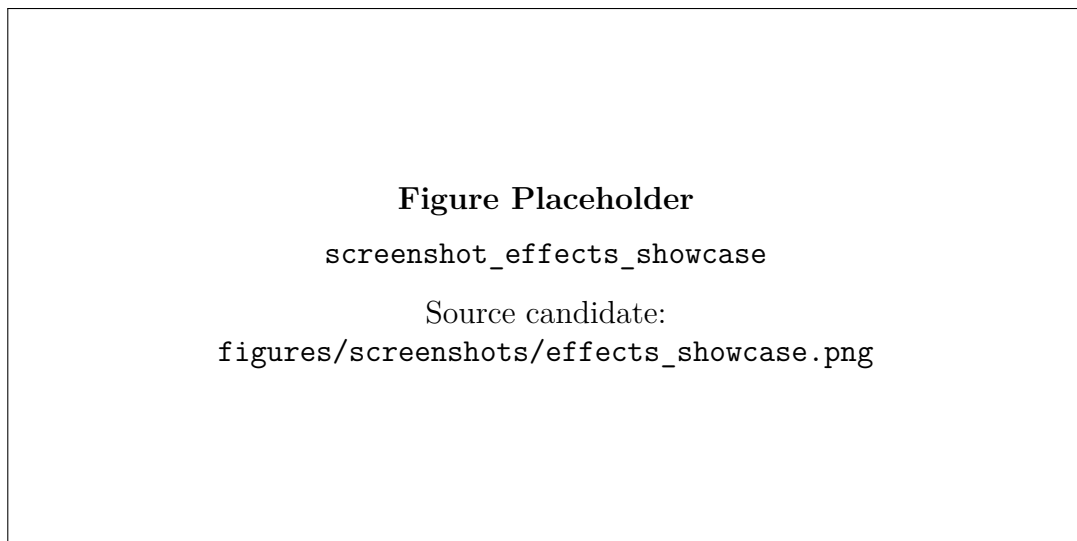


Figure 6.2: Effects showcase screenshot placeholder (fire, lightning, rasengan, water dragon, kuchiyose).

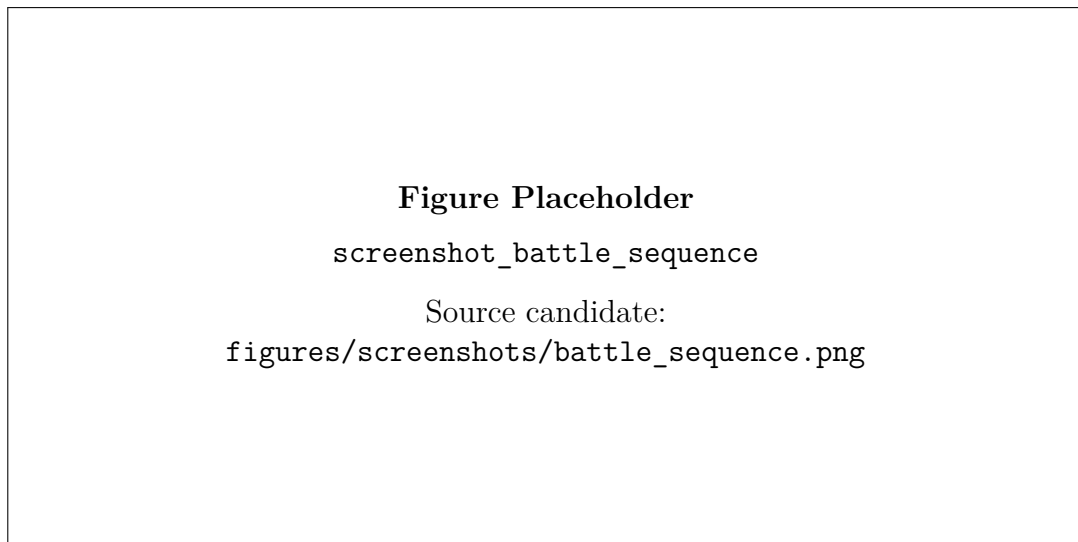


Figure 6.3: Battle sequence progress screenshot placeholder.

Chapter 7

Limitations and Future Work

7.1 Current Limitations

7.1.1 Dataset and Labeling

- Class imbalance remains visible (for example, tiger vs boar support counts).
- Background and lighting variation may still be limited compared to production use.
- Label naming mismatch (hare vs rabbit aliasing) adds integration complexity.

7.1.2 Model and Inference

- RandomForest operates on per-frame static features and does not model temporal dynamics directly.
- Confidence thresholds require manual tuning per runtime environment.
- Similar hand poses can still produce occasional confusion.

7.1.3 iOS Runtime and UX

- Performance and stability may vary by device generation and camera quality.
- Effects and tracking quality are sensitive to fast motion blur and partial occlusion.
- Guidance timing and hold duration messaging should remain synchronized with logic constants.

7.2 Recommended Improvements

1. Add temporal modeling (sequence-aware classifier or lightweight recurrent model).
2. Expand training data with multi-user and multi-environment captures.
3. Automate threshold calibration and per-class confidence analysis.
4. Add model versioning and reproducible experiment logging.

5. Build an automated evaluation suite for on-device regression testing.
6. Integrate screenshot and telemetry capture for documentation updates.

7.3 Documentation Maintenance Plan

- Update metrics tables after each model retrain.
- Replace placeholders with latest app screenshots before submission.
- Keep jutsu-sequence table aligned with `AppModels.swift` source changes.

Appendix A

Expected Screenshot Files

Place screenshots in the following paths and recompile:

- `figures/screenshots/home_view.png`
- `figures/screenshots/mode_setup.png`
- `figures/screenshots/tutorial_overlay.png`
- `figures/screenshots/battle_hud.png`
- `figures/screenshots/runtime_prediction.png`
- `figures/screenshots/effects_showcase.png`
- `figures/screenshots/confusion_matrix_heatmap.png`